Actividad 1 (Velocidades Lineales y angulares)

Edwin Emmanuel Salazar Meza A01738380

16 de febrero de 2026

Fundamentación de robótica

Gpo 101

Edwin Emmanuel Salazar Meza A01738380
Fundamentación de robótica

Introducción

Este trabajo modela un robot planar de 3 GDL en MATLAB para el análisis cinemático y de velocidades. Permite el cálculo automático de matrices de transformación, Jacobianos y vectores de velocidad (lineal y angular), esenciales para la precisión en el desplazamiento y posicionamiento.

Descripción

Se presenta el análisis de un robot manipulador planar de 3 grados de libertad (GDL).
Las características principales de este robot son:

- **Articulaciones:** Todas son de tipo rotacional.
- **Ejes de Giro:** Son completamente paralelos al eje z.
- **Movimiento:** La cinemática del robot se desarrolla exclusivamente en el plano XY.

Coordenadas generalizadas Velocidades generalizadas

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix} \qquad \mathbf{q} = \begin{bmatrix} \theta_1(t) \\ \theta_2(t) \\ \theta_3(t) \end{bmatrix}$$

Metodología

Primero se obtienen las matrices de transformación homogénea para cada eslabón.

$$A_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & l_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i & 0 & l_i\sin\theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Para obtener la matriz de transformación global siendo el resultado de:

$$T3 = A1 * A2 * A3$$

$$Donde\ A1\ \ transforma\ del\ marco\ 0\ al\ 1$$

$$Donde\ A2\ \ transforma\ del\ marco\ 1\ al\ 2$$

$$Donde\ A3\ \ transforma\ del\ marco\ 2\ al\ 3$$

$$J_\omega = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Por último se calcula la velocidad lineal y angular:

$$\mathbf{v} = J_v \dot{\mathbf{q}} \qquad \boldsymbol{\omega} = J_\omega \dot{\mathbf{q}}$$

Dando los siguientes vectores:

$$\mathbf{v} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ 0 \end{bmatrix} \qquad \boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 + \dot{\theta}_2 + \dot{\theta}_3 \end{bmatrix}$$

Código MATLAB

```
%Declaración de variables simbólicas
syms th1(t) th2(t) th3(t) t l1 l2 l3

%Configuración del robot, 0 para junta rotacional, 1 para junta prismática
RP=[0 0 0];

%Creamos el vector de coordenadas articulares
Q= [th1, th2, th3];
disp('Coordenadas generalizadas');
pretty (Q);

%Creamos el vector de velocidades generalizadas
Qp= diff(Q, t);
disp('Velocidades generalizadas');
pretty (Qp);
%Número de grado de libertad del robot
GDL= size(RP,2);
GDL_str= num2str(GDL);
```

El código inicia preparando el entorno de trabajo y definiendo variables simbólicas dependientes del tiempo. Esto permite analizar el movimiento del robot de manera analítica.

Se establecen las posiciones locales y las matrices de rotación específicas para cada articulación. A continuación, se inicializan las matrices de transformación homogénea, tanto a nivel local como global, junto con las posiciones y rotaciones observadas desde el sistema de referencia inercial.

```
%Junta 1
%Posición de la junta 1 respecto a 0
P(:,:,1)= [l1*cos(th1); l1*sin(th1);0];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,1)= [cos(th1) -sin(th1)  0;
           sin(th1)  cos(th1)  0;
           0         0         1];


%Junta 2
%Posición de la junta 1 respecto a 0
P(:,:,2)= [l2*cos(th2); l2*sin(th2);0];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,2)= [cos(th2) -sin(th2)  0;
           sin(th2)  cos(th2)  0;
           0         0         1];


%Junta 3
%Posición de la junta 1 respecto a 0
P(:,:,3)= [l3*cos(th3); l3*sin(th3);0];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,3)= [cos(th3) -sin(th3)  0;
           sin(th3)  cos(th3)  0;
           0         0         1];


%Creamos un vector de ceros
Vector_Zeros= zeros(1, 3);

%Inicializamos las matrices de transformación Homogénea locales
A(:,:,GDL)=simplify([R(:,:,GDL) P(:,:,GDL); Vector_Zeros 1]);
%Inicializamos las matrices de transformación Homogénea globales
T(:,:,GDL)=simplify([R(:,:,GDL) P(:,:,GDL); Vector_Zeros 1]);
%Inicializamos las posiciones vistas desde el marco de referencia inercial
PO(:,:,GDL)= P(:,:,GDL);
%Inicializamos las matrices de rotación vistas desde el marco de referencia inercial
RO(:,:,GDL)= R(:,:,GDL);
%Inicializamos las INVERSAS de las matrices de rotación vistas desde el marco de referencia inercial
RO_inv(:,:,GDL)= R(:,:,GDL);
```

Cada articulación se itera para calcular la transformación global acumulada y, a partir del marco base, se extraen las correspondientes posiciones y rotaciones.

```
for i = 1:GDL
    i_str= num2str(i);
    %Locales
    disp(strcat('Matriz de Transformación local A', i_str));
    A(:,:,i)=simplify([R(:,:,i) P(:,:,i); Vector_Zeros 1]);
    pretty (A(:,:,i));

   %Globales
    try
       T(:,:,i)= T(:,:,i-1)*A(:,:,i);
    catch
       T(:,:,i)= A(:,:,i);
    end
    disp(strcat('Matriz de Transformación global T', i_str));
    T(:,:,i)= simplify(T(:,:,i));
    pretty(T(:,:,i))

    RO(:,:,i)= T(1:3,1:3,i);
    RO_inv(:,:,i)= transpose(RO(:,:,i));
    PO(:,:,i)= T(1:3,4,i);
    %pretty(RO(:,:,i));
    %pretty(RO_inv(:,:,i));
    %pretty(PO(:,:,i));
end
```

Se procede al cálculo del Jacobiano lineal mediante derivación directa y un enfoque analítico. Además, se determina el Jacobiano angular analítico, culminando con la obtención de los vectores de velocidad lineal y angular.

```
%Calculamos el jacobiano lineal de forma diferencial
disp('Jacobiano lineal obtenido de forma diferencial');
%Derivadas parciales de x respecto a th1 y th2
Jv11= functionalDerivative(PO(1,1,GDL), th1);
Jv12= functionalDerivative(PO(1,1,GDL), th2);
Jv13= functionalDerivative(PO(1,1,GDL), th3);
%Derivadas parciales de y respecto a th1 y th2
Jv21= functionalDerivative(PO(2,1,GDL), th1);
Jv22= functionalDerivative(PO(2,1,GDL), th2);
Jv23= functionalDerivative(PO(2,1,GDL), th3);
%Derivadas parciales de z respecto a th1 y th2
Jv31= functionalDerivative(PO(3,1,GDL), th1);
Jv32= functionalDerivative(PO(3,1,GDL), th2);
Jv33= functionalDerivative(PO(3,1,GDL), th3);

%Creamos la matríz del Jacobiano lineal
jv_d=simplify([Jv11 Jv12 Jv13;
               Jv21 Jv22 Jv23;
               Jv31 Jv32 Jv33]);
pretty(jv_d);


%Calculamos el jacobiano lineal de forma analítica
Jv_a(:,GDL)=PO(:,:,GDL);
Jw_a(:,GDL)=PO(:,:,GDL);

for k= 1:GDL
    if RP(k)==0 %Casos: articulación rotacional
       %Para las juntas de revolución
        try
            Jv_a(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL)-PO(:,:,k-1));
            Jw_a(:,k)= RO(:,3,k-1);
        catch
            Jv_a(:,k)= cross([0,0,1], PO(:,:,GDL));
            Jw_a(:,k)=[0,0,1];
        end

        %Para las juntas prismáticas
     elseif RP(k)==1 %Casos: articulación prismática
%
        try
            Jv_a(:,k)= RO(:,3,k-1);
        catch
            Jv_a(:,k)=[0,0,1];
        end
            Jw_a(:,k)=[0,0,0];
     end
 end

Jv_a= simplify (Jv_a);
Jw_a= simplify (Jw_a);
disp('Jacobiano lineal obtenido de forma analítica');
pretty (Jv_a);
disp('Jacobiano ángular obtenido de forma analítica');
pretty (Jw_a);


disp('Velocidad lineal obtenida mediante el Jacobiano lineal');
V=simplify (Jv_a*Qp');
pretty(V);
disp('Velocidad angular obtenida mediante el Jacobiano angular');
W=simplify (Jw_a*Qp');
pretty(W);
```